

1. 일반 함수와 람다식의 차이

① 일반 함수

일반 함수는 특정 작업을 수행하도록 이름을 붙여 정의한 코드 블록이다. 함수는 프로그램의 여러 위치에서 호출하여 재사용할 수 있으며, 일반적으로 반환형, 함수 이름, 매개변수, 함수 본문으로 구성된다.

② 람다식(Lambda Expression)

람다식은 함수의 이름을 별도로 정의하지 않고 필요한 위치에서 직접 작성할 수 있는 익명 함수 객체를 만드는 표현식이다. C++11부터 지원되며, 간단한 기능을 짧은 코드로 구현할 때 유용하다.

람다식의 기본 형식은 `[캡처](매개변수) { 함수 본문 }`이다.

1-2. 일반 함수와 람다식의 코드 비교

① 일반 함수 예제

```
```cpp
#include <iostream>
using namespace std;

int add(int a, int b) {
 return a + b;
}

int main() {
 cout << add(3, 5) << endl;
 return 0;
}
```
```

실행 결과: 8

일반 함수는 `add()`라는 이름으로 함수를 정의하고, `main()` 함수에서 호출하여 사용한다.

② 람다식 예제

```

```cpp
#include <iostream>
using namespace std;

int main() {
 auto add = [](int a, int b) {
 return a + b;
 };

 cout << add(3, 5) << endl;
 return 0;
}
```

```

실행 결과: 8

람다식은 별도의 일반 함수를 정의하지 않고 `main()` 함수 내부에서 필요한 기능을 직접 작성한다.

1-3. 일반 함수와 람다식의 주요 차이점

| 구분 | 일반 함수 | 람다식 |
|----------|-------------------------|------------------------|
| 함수 이름 | 일반적으로 이름이 있음 | 익명 함수 객체 생성 |
| 정의 위치 | 독립적으로 정의 가능 | 표현식이 허용되는 위치에서 작성 가능 |
| 코드 길이 | 별도 함수 정의 필요 | 간단한 기능을 짧게 작성 가능 |
| 외부 변수 사용 | 매개변수 전달 또는 접근 가능한 변수 사용 | 캡처 기능으로 주변 지역 변수 사용 가능 |
| 재사용성 | 여러 위치에서 재사용하기 편리 | 주로 특정 위치에서 간단한 작업에 사용 |
| 주요 용도 | 반복적으로 사용하는 기능 | 일회성 작업, 콜백 함수, 알고리즘 |

1-4. 일반 함수 대신 람다식을 사용할 때의 장점

① 코드의 간결성

별도의 함수를 정의할 필요 없이 필요한 위치에서 직접 기능을 구현할 수 있어 코드가 간결해진다.

② 가독성 향상

간단한 작업을 사용하는 위치에 바로 작성할 수 있으므로 코드의 흐름을 이해하기 쉽다.

③ 외부 변수 캡처 기능

람다식은 캡처 기능을 통해 주변 지역 변수를 값이나 참조 형태로 사용할 수 있다. 따라서 별도의 매개변수 전달 없이도 필요한 데이터를 활용할 수 있다.

④ 일회성 함수 작성에 유리

한 번만 사용하는 간단한 기능을 위해 별도의 일반 함수를 정의할 필요가 없어 편리하다.

⑤ STL 알고리즘과 함께 사용하기 편리

`sort()`, `for\_each()`, `find\_if()` 등의 STL 알고리즘에서 람다식을 사용하면 정렬 조건이나 처리 동작을 간단하게 지정할 수 있다.

1-5. 결론

일반 함수와 람다식은 모두 특정 작업을 수행하는 데 사용되지만, 정의 방식과 활용 목적에 차이가 있다.

일반 함수는 여러 곳에서 반복적으로 사용하는 기능에 적합하고, 람다식은 특정 위치에서 간단한 작업을 수행하거나 외부 변수를 활용해야 할 때 유용하다.

따라서 반복적으로 사용하는 기능은 일반 함수로 구현하고, 간단한 일회성 작업이나 STL 알고리즘의 조건 지정에는 람다식을 사용하는 것이 효과적이다.

2 인터넷에서 찾은 예제

참고 자료는 C++ 공식 레퍼런스로 널리 사용되는 cppreference의 `std::function` 설명. 여기에는 일반 함수와 람다식을 `std::function`에 저장하고 다른 함수에 전달하는 예제가 있음.

cppreference.com

+1

- `std::function` 참고 자료

- C++ 람다식 참고 자료

2-2 C++ 실습 소스 코드와 실행 결과

```
1
2 #include <iostream>
3 #include <vector>
4 #include <functional>
5 using namespace std;
6
7 // 전달받은 함수를 각 숫자에 적용
8 void process(const vector<int>& vec, function<void(int)> func) {
9     for (int num : vec) {
10         func(num);
11     }
12 }
13
14 // 숫자를 출력하는 일반 함수
15 void print(int num) {
16     cout << num << " ";
17 }
18
19 int main() {
20     vector<int> vec = {1, 2, 3, 4, 5};
21
22     // 일반 함수를 인자로 전달
23     process(vec, print);
24     cout << endl;
25
26     // 람다식을 인자로 전달
27     int sum = 0;
28     process(vec, [&sum](int num) {
29         sum += num;
30     });
31
32     cout << "sum=" << sum << endl;
33     return 0;
34 }
35
```

1 2 3 4 5
sum=15

2-3. 소스 코드 자세한 설명

실습 목적

C++의 `std::function`과 람다식을 이용하여 함수를 다른 함수의 인자로 전달하는 방법을 이해한다. 일반 함수와 람다식을 각각 인자로 전달하여 실행 결과를 비교한다.

소스 코드 설명

① 헤더 파일 선언

- `#include <iostream>` : 화면에 데이터를 출력하기 위한 헤더 파일이다.
- `#include <vector>` : 여러 정수를 저장하는 `vector` 컨테이너를 사용하기 위한 헤더 파일이다.
- `#include <functional>` : `std::function`을 사용하기 위한 헤더 파일이다.

- ``using namespace std;`` : `std::`를 생략하고 표준 라이브러리 기능을 사용할 수 있게 한다.

② process 함수 정의

```
`void process(const vector<int>& vec, function<void(int)> func)`
```

정수 벡터와 함수를 인자로 전달받는 함수이다.

- ``const vector<int>& vec`` : 벡터를 복사하지 않고 참조로 전달받으며, 함수 내부에서 수정하지 못하도록 한다.
- ``function<void(int)> func`` : 정수 하나를 전달받고 반환값이 없는 함수를 저장하여 호출할 수 있는 객체이다.
- ``for (int num : vec)`` : 벡터에 저장된 숫자를 하나씩 가져온다.
- ``func(num)`` : 전달받은 함수를 각 숫자에 대해 실행한다.

③ print 함수 정의

```
`void print(int num)`
```

전달받은 정수를 화면에 출력하는 일반 함수이다.

``cout << num << " ";``는 숫자와 공백을 출력한다.

④ main 함수와 벡터 생성

```
`vector<int> vec = {1, 2, 3, 4, 5};`
```

1부터 5까지의 정수를 저장하는 벡터를 생성한다.

⑤ 일반 함수를 인자로 전달

```
`process(vec, print);`
```

process 함수에 벡터와 일반 함수 print를 전달한다.

process 함수는 벡터의 각 숫자를 print 함수에 전달하므로 1 2 3 4 5가 순서대로 출력된다.

⑥ 람다식을 인자로 전달

```
```cpp
```

```
int sum = 0;
process(vec, [&sum](int num) {
 sum += num;
});
```

```

- `int sum = 0;` : 숫자의 합계를 저장할 변수를 선언하고 0으로 초기화한다.
- `[&sum]` : 외부 변수 `sum`을 참조 방식으로 캡처하여 람다식 내부에서 변경할 수 있게 한다.
- `(int num)` : `process` 함수에서 전달한 정수를 받는 매개변수이다.
- `sum += num;` : 전달받은 숫자를 `sum`에 누적하여 더한다.

따라서 $1+2+3+4+5$ 가 계산되어 `sum`에는 15가 저장된다.

⑦ 최종 결과 출력

```
`cout << "sum=" << sum << endl;`
```

계산된 합계를 화면에 출력한다.

실행 결과

```
```text
1 2 3 4 5
sum=15
```
```

실습 결과 및 고찰

이번 실습에서는 `std::function`을 사용하여 일반 함수와 람다식을 동일한 함수의 인자로 전달하였다.

일반 함수 `print`를 전달했을 때는 벡터의 숫자가 출력되었고, 람다식을 전달했을 때는 벡터의 숫자를 모두 더한 결과가 출력되었다.

특히 람다식의 참조 캡처 기능을 통해 외부 변수 `sum`의 값을 직접 변경할 수 있다는 점을 확인하였다.

이를 통해 `std::function`을 사용하면 하나의 함수에서 서로 다른 동작을 유연하게 수행할 수 있으며, 람다식을 이용하면 별도의 함수를 정의하지 않고 필요한 기능을 간결하게 구현할 수 있음을 알 수 있었다.

3. 함수 포인터와 std::function 객체의 차이

1. 함수 포인터(Function Pointer)란?

함수 포인터는 함수가 저장된 메모리 주소를 가리키는 포인터이다. 일반적인 포인터가 변수의 주소를 저장한다면, 함수 포인터는 함수의 주소를 저장하여 해당 함수를 간접적으로 호출할 수 있도록 한다.

함수 포인터를 사용하면 실행할 함수를 프로그램 실행 중에 선택하거나, 함수를 다른 함수의 인자로 전달할 수 있다.

함수 포인터의 특징

- 함수의 메모리 주소를 저장한다.
- 반환형과 매개변수 형식이 일치하는 함수를 가리킬 수 있다.
- 일반 함수와 static 멤버 함수 등을 연결할 수 있다.
- 캡처가 있는 람다식은 일반 함수 포인터로 직접 변환할 수 없다.
- 구조가 단순하여 일반적으로 호출 오버헤드가 작다.

함수 포인터 예제

```
```cpp
#include <iostream>
using namespace std;

void hello() {
 cout << "Hello C++" << endl;
}

int main() {
 void (*funcPtr)() = hello;
 funcPtr();

 return 0;
}
```
```

실행 결과:

```
```text
Hello C++
```

...

## 코드 설명

`void hello()`는 문자열을 출력하는 일반 함수이다.

`void (*funcPtr)() = hello;`는 반환형이 `void`이고 매개변수가 없는 함수를 가리키는 함수 포인터를 선언한 것이다.

`funcPtr()`를 실행하면 함수 포인터가 가리키는 `hello` 함수가 호출된다.

## 2. `std::function` 객체란?

`std::function`은 C++11부터 제공되는 표준 라이브러리의 함수 래퍼 클래스 템플릿이다.

함수 포인터와 달리 일반 함수뿐만 아니라 람다식, 함수 객체 등 다양한 호출 가능한 객체를 저장하고 실행할 수 있다.

`std::function`을 사용하려면 `<functional>` 헤더 파일을 포함해야 한다.

## `std::function`의 특징

- 일반 함수와 함수 포인터를 저장할 수 있다.
- 캡처가 있는 람다식도 저장할 수 있다.
- 함수 객체(Functor)를 저장할 수 있다.
- 서로 다른 종류의 호출 가능한 객체를 동일한 형식으로 관리할 수 있다.
- 함수 포인터보다 유연하지만 추가적인 메모리 및 실행 비용이 발생할 수 있다.

## `std::function` 예제

```
```cpp
#include <iostream>
#include <functional>
using namespace std;

int main() {
    int num = 10;

    function<void()> func = [num]() {
        cout << "num = " << num << endl;
    };
}
```

```

func();

return 0;
}
'''

```

실행 결과:

```

```text
num = 10
'''

```

코드 설명

``int num = 10;``은 정수형 변수 `num`을 선언한다.

``function<void()> func``는 매개변수가 없고 반환값이 없는 호출 가능한 객체를 저장하는 `std::function` 객체를 선언한다.

``[num]()``은 외부 변수 `num`을 값으로 캡처하는 람다식이다.

``func()``를 실행하면 저장된 람다식이 호출되어 `num`의 값인 10이 출력된다.

### 3. 함수 포인터와 `std::function`의 주요 차이점

구분	함수 포인터	<code>std::function</code>
정의	함수의 주소를 저장하는 포인터	호출 가능한 객체를 저장하는 클래스 템플릿
헤더 파일	별도 헤더 불필요	<code>&lt;functional&gt;</code> 필요
일반 함수	사용 가능	사용 가능
캡처 없는 람다식	사용 가능	사용 가능
캡처 있는 람다식	직접 저장 불가능	사용 가능
함수 객체	직접 저장 불가능	사용 가능
메모리 사용	일반적으로 적음	상대적으로 많을 수 있음
실행 속도	일반적으로 빠름	추가 호출 비용이 발생할 수 있음
유연성	상대적으로 낮음	높음
사용 목적	단순한 함수 호출 및 전달	다양한 호출 객체의 통합 관리

### 4. 함수 포인터와 `std::function`의 장단점

#### ① 함수 포인터의 장점

함수의 주소만 저장하므로 구조가 단순하고 메모리 사용량이 적다. 일반적으로 호출 비용이 작아 성능이 중요한 프로그램에 적합하다.

## ② 함수 포인터의 단점

캡처가 있는 람다식이나 일반적인 함수 객체를 직접 저장할 수 없어 활용 범위가 제한적이다.

## ③ std::function의 장점

일반 함수, 람다식, 함수 객체 등 다양한 호출 가능한 객체를 하나의 형식으로 관리할 수 있다. 특히 람다식의 캡처 기능을 활용할 수 있어 복잡한 동작을 유연하게 구현할 수 있다.

## ④ std::function의 단점

함수 포인터에 비해 내부 구조가 복잡하고, 객체 저장과 호출 과정에서 추가적인 메모리 사용 및 실행 비용이 발생할 수 있다.

## 5. 결론

함수 포인터와 std::function은 모두 함수를 간접적으로 호출하거나 다른 함수에 전달하기 위해 사용된다.

함수 포인터는 구조가 단순하고 메모리 사용량이 적어 일반 함수를 효율적으로 호출하는 데 적합하다.

반면 std::function은 일반 함수뿐만 아니라 람다식과 함수 객체까지 저장할 수 있어 유연성과 확장성이 뛰어나다.

따라서 단순한 함수 호출이나 성능이 중요한 경우에는 함수 포인터를 사용하고, 다양한 호출 가능한 객체를 관리하거나 캡처가 있는 람다식을 사용해야 하는 경우에는 std::function을 사용하는 것이 적합하다.

## 4. 23페이지 예제처럼 인터넷에서 std::function 클래스를 함수에 매개 변수에 사용한 예제를 찾아서 실행해보고 소스코드를 자세히 설명 하시오.

### 1. 인터넷 참고 자료

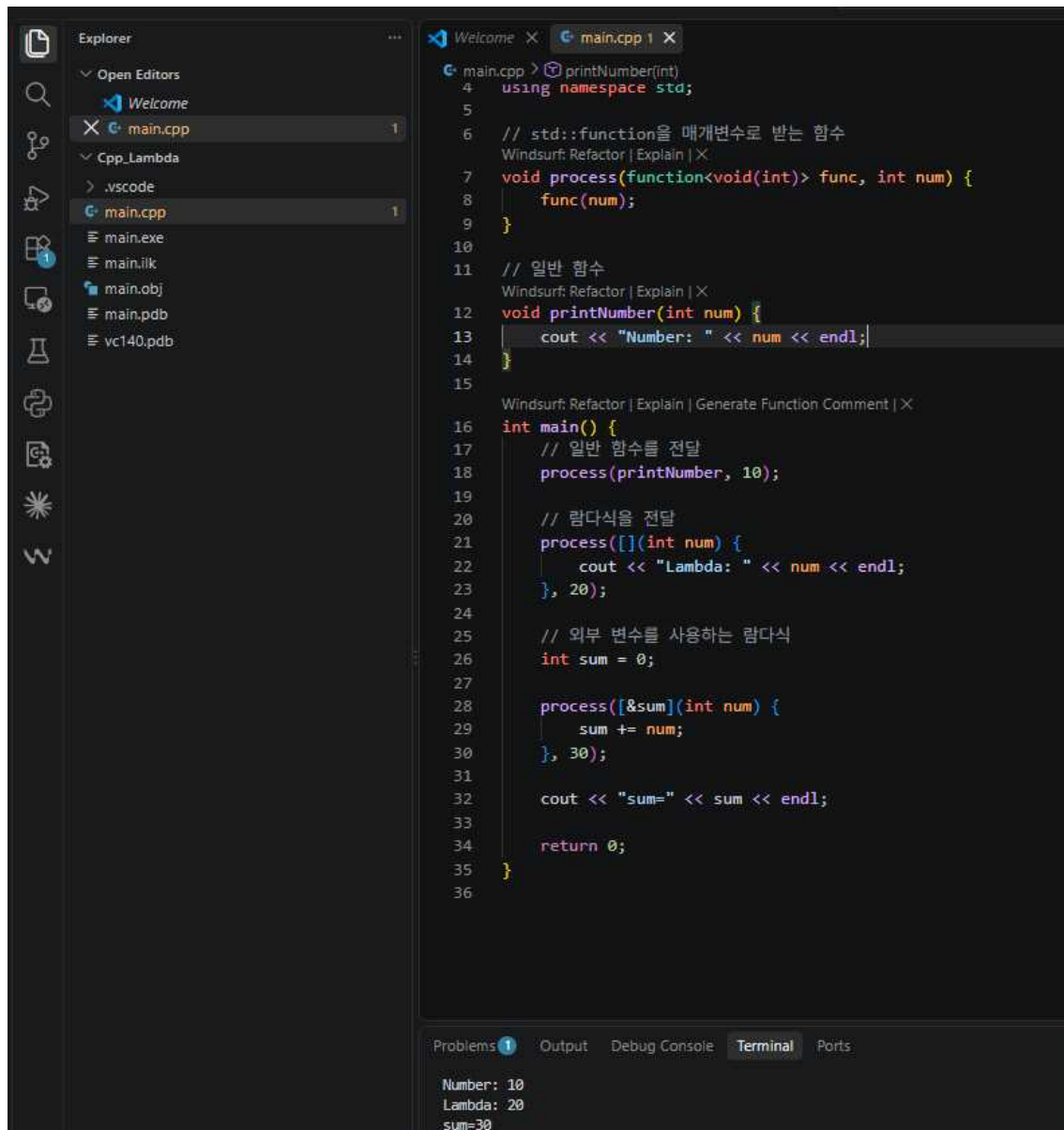
참고 사이트: [cppreference - std::function::operator\(\)](#)

이 페이지에서는 `std::function`을 함수의 매개변수로 전달하고, 일반 함수와 람다식을 호출하는 예제를 설명하고 있음.

Cppreference

아래 코드는 이 자료를 참고

## 2. C++ 실습 소스 코드와 실행 결과



The screenshot shows a C++ IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'Cpp\_Lambda' with files 'main.cpp', 'main.exe', 'main.ilc', 'main.obj', 'main.pdb', and 'vc140.pdb'. The code editor shows the following C++ code:

```
1 // std::function을 매개변수로 받는 함수
2 using namespace std;
3
4 // std::function을 매개변수로 받는 함수
5 Windsurf: Refactor | Explain | X
6 void process(function<void(int)> func, int num) {
7 func(num);
8 }
9
10 // 일반 함수
11 Windsurf: Refactor | Explain | X
12 void printNumber(int num) {
13 cout << "Number: " << num << endl;
14 }
15
16 Windsurf: Refactor | Explain | Generate Function Comment | X
17 int main() {
18 // 일반 함수를 전달
19 process(printNumber, 10);
20
21 // 람다식을 전달
22 process([](int num) {
23 cout << "Lambda: " << num << endl;
24 }, 20);
25
26 // 외부 변수를 사용하는 람다식
27 int sum = 0;
28
29 process([&sum](int num) {
30 sum += num;
31 }, 30);
32
33 cout << "sum=" << sum << endl;
34
35 return 0;
36 }
```

The terminal output at the bottom shows the following results:

```
Number: 10
Lambda: 20
sum=30
```

## 4. 소스 코드 자세한 설명

C++ 고급기능 실습과제

`std::function` 클래스를 함수의 매개변수로 사용하는 예제

## 1. 실습 목적

C++의 `std::function` 클래스를 함수의 매개변수로 사용하는 방법을 이해하고, 일반 함수와 람다식을 다른 함수에 전달하여 실행하는 과정을 확인한다.

## 2. 소스 코드 설명

### ① 헤더 파일 선언

```
`#include <iostream>`
```

화면에 데이터를 출력하기 위한 표준 입출력 헤더 파일이다.

```
`#include <functional>`
```

`std::function` 클래스를 사용하기 위한 헤더 파일이다.

```
`using namespace std;`
```

표준 라이브러리의 기능을 사용할 때 `std::`를 생략할 수 있도록 한다.

### ② process 함수 정의

```
```cpp
void process(function<void(int)> func, int num) {
    func(num);
}
```
```

`process` 함수는 `std::function` 객체와 정수형 변수를 매개변수로 전달받는다.

- `function<void(int)> func` : 정수형 매개변수 하나를 받고 반환값이 없는 호출 가능한 객체를 전달받는다.
- `int num` : 함수에 전달할 정수 값을 저장한다.
- `func(num)` : 전달받은 함수 또는 람다식을 호출하면서 `num` 값을 전달한다.

따라서 `process` 함수는 전달받은 함수에 따라 서로 다른 작업을 수행할 수 있다.

### ③ 일반 함수 정의

```
```cpp
```

```
void printNumber(int num) {
    cout << "Number: " << num << endl;
}
...

```

정수형 매개변수를 전달받아 화면에 출력하는 일반 함수이다.

④ 일반 함수를 매개변수로 전달

```
`process(printNumber, 10);`

```

process 함수에 일반 함수 printNumber와 정수 10을 전달한다.

process 내부에서 printNumber(10)이 호출되므로 Number: 10이 출력된다.

⑤ 람다식을 매개변수로 전달

```
```cpp
process([](int num) {
 cout << "Lambda: " << num << endl;
}, 20);
...

```

별도로 이름을 가진 함수를 정의하지 않고 람다식을 직접 전달한다.

- `[]` : 외부 변수를 캡처하지 않는다는 의미이다.
- `(int num)` : 정수형 매개변수를 전달받는다.
- `cout` : 전달받은 정수를 출력한다.

따라서 Lambda: 20이 출력된다.

#### ⑥ 외부 변수를 사용하는 람다식

```
```cpp
int sum = 0;

process([&sum](int num) {
    sum += num;
}, 30);
...

```

먼저 sum 변수를 0으로 초기화한다.

``[&sum]``은 외부 변수 sum을 참조 방식으로 캡처한다는 의미이다.

``sum += num``은 전달받은 정수 30을 sum에 더한다.

따라서 sum의 값은 30으로 변경된다.

⑦ 최종 결과 출력

```
`cout << "sum=" << sum << endl;`
```

람다식에서 변경한 sum 값을 화면에 출력한다.

3. 실행 결과

```
```text
Number: 10
Lambda: 20
sum=30
```
```

4. 실습 결과 및 고찰

이번 실습에서는 `std::function`을 함수의 매개변수로 선언하고, 일반 함수와 람다식을 전달하여 실행하는 방법을 학습하였다.

일반 함수를 전달했을 때는 정수를 출력하였고, 람다식을 전달했을 때는 다른 형식으로 정수를 출력하였다.

또한 참조 캡처 기능을 사용하는 람다식을 전달하여 외부 변수의 값을 변경할 수 있음을 확인하도록 구성하였다.

이를 통해 `std::function`을 사용하면 하나의 함수에서 다양한 호출 가능한 객체를 전달받아 실행할 수 있으며, 프로그램의 유연성과 재사용성을 높일 수 있다는 점을 이해하였다.

5. 참고 자료

cppreference - `std::function::operator()`

<https://cppreference.com/cpp/utility/functional/function/operator/>